Ubuntu Autoinstall

Storage Configuration Complete Reference

`autoinstall.yaml → storage:`

Ubuntu 20.04 LTS and later (Subiquity installer)

1. Overview

The storage section of autoinstall.yaml defines the entire disk layout for an unattended Ubuntu installation. It is a declarative specification that maps directly to the curtin storage configuration schema. Every physical disk, partition, RAID array, LVM group, filesystem, and mount point must be declared explicitly as a list of storage objects.

The top-level structure is:

```
autoinstall:
  version: 1
  storage:
    layout:          # Optional shorthand (simple cases only)
      name: lvm | direct | zfs
    config:          # Explicit object list (full control)
      - id: disk0
        type: disk
        ...
```

NOTE: Use config: for production systems. The layout: shorthand does not support custom partition sizes, RAID, LVM with custom VGs, or multiple disks.

2. Storage Object Types

Every entry in config: has a required type field. The supported types are:

Type	Maps To	Purpose
disk	Physical block device	Identifies a disk; initializes partition table
partition	Partition on a disk	Carves space from a disk object
raid	Linux MD RAID	Combines disks/partitions into a RAID array
lvm_volgroup	LVM Volume Group (VG)	Groups physical volumes

lvm_partition	LVM Logical Volume (LV)	Allocates space from a VG
bcache	bcache device	SSD caching layer over HDD
format	Filesystem	Creates a filesystem on any block device
mount	Mount point	Mounts a formatted device into the filesystem tree

3. type: disk

A disk object represents a physical block device. It must appear before any partition or other object that references it.

3.1 All Fields

Field	Type	Required	Description
id	string	Yes	Unique identifier referenced by other objects
type	string	Yes	Must be "disk"
ptable	string	No	Partition table type: gpt (default) or msdos
match	mapping	No	Selector to identify the physical disk (see below)
path	string	No	Explicit path e.g. /dev/sda (alternative to match)
serial	string	No	Disk serial number for matching
wwn	string	No	World Wide Name for matching NVMe/SAS disks
preserve	boolean	No	true = keep existing partition table; default false
name	string	No	Udev name alias for the disk
grub_device	boolean	No	true = install GRUB bootloader to this disk
wipe	string	No	Wipe strategy: superblock, zero, random, or null
nvme_controller	string	No	Reference to nvme_controller object ID

3.2 match Object

The match mapping lets the installer identify a disk by attributes rather than a fixed path, which is more robust across hardware changes.

match key	Meaning
size: largest	Select the largest available disk

size: smallest	Select the smallest available disk
model: <pattern>	Match disk model string (glob supported)
serial: <value>	Match exact serial number
path: <pattern>	Match device path pattern e.g. /dev/nvme*
id: <value>	Match udev ID_PATH attribute
ssd: true|false	Match only SSDs or only HDDs

3.3 Examples

Select the largest disk and initialize GPT:

```
- id: root_disk
  type: disk
  ptable: gpt
  match:
    size: largest
  grub_device: true
  wipe: superblock
```

Select a specific disk by serial number:

```
- id: data_disk
  type: disk
  ptable: gpt
  match:
    serial: WD-WX31A12B3456
```

Select by path pattern (e.g. first NVMe):

```
- id: nvme0
  type: disk
  ptable: gpt
  match:
    path: /dev/nvme0n1
```

4. type: partition

A partition object creates a partition on a disk. Partitions are created in the order they appear in the config list.

4.1 All Fields

Field	Type	Required	Description
id	string	Yes	Unique identifier
type	string	Yes	Must be "partition"
device	string	Yes	ID of the parent disk object
size	string/int	Yes*	Partition size. Use suffix: G, M, T, or "-1" for remainder
number	integer	No	Explicit partition number (auto-assigned if omitted)
flag	string	No	GPT/MBR flags: boot, bios_grub, swap, home, extended, logical
preserve	boolean	No	true = do not modify this partition
wipe	string	No	Wipe strategy before use: superblock, zero, random
name	string	No	GPT partition name label
grub_device	boolean	No	true = install GRUB to this partition
resize	boolean	No	true = resize partition (only when preserve: true)
offset	integer	No	Start offset in bytes (advanced, rarely needed)

4.2 Size Specification

Value	Meaning
512M	512 Mebibytes
50G	50 Gibibytes
2T	2 Tebibytes
-1	Use all remaining space on the disk
50%	50% of the disk (percentage support varies by version)

NOTE: Always create the EFI System Partition (ESP) with flag: boot on GPT disks. On BIOS/legacy systems, create a 1 MiB partition with flag: bios_grub instead.

4.3 Partition Flag Reference

Flag	Applies To	Effect
boot	GPT	Sets EFI System Partition type GUID — required for UEFI boot
bios_grub	GPT	Sets BIOS Boot Partition type GUID — required for legacy GRUB on GPT
swap	MBR/GPT	Marks partition as Linux swap type

home	MBR/GPT	Marks partition as Linux home type (informational)
extended	MBR	Creates MBR extended partition container
logical	MBR	Creates logical partition inside extended

4.4 Examples

UEFI partitioning layout (GPT + EFI):

```
- id: efi_part
  type: partition
  device: root_disk
  size: 512M
  flag: boot
  grub_device: true

- id: root_part
  type: partition
  device: root_disk
  size: -1
```

Legacy BIOS layout (GPT with bios_grub):

```
- id: bios_part
  type: partition
  device: root_disk
  size: 1M
  flag: bios_grub

- id: boot_part
  type: partition
  device: root_disk
  size: 1G

- id: root_part
  type: partition
  device: root_disk
  size: -1
```

5. type: format

A format object creates a filesystem on a block device (partition, LVM logical volume, RAID array, etc.).

5.1 All Fields

Field	Type	Required	Description
id	string	Yes	Unique identifier (referenced by mount objects)
type	string	Yes	Must be "format"
volume	string	Yes	ID of the block device to format
fstype	string	Yes	Filesystem type (see table below)
label	string	No	Filesystem label
uuid	string	No	Explicit UUID to assign (advanced)
preserve	boolean	No	true = do not reformat; mount existing filesystem

5.2 Filesystem Types (fstype)

fstype	Use Case	Notes
ext4	Root, home, data	Default Linux filesystem; journaled; widely supported
vfat	EFI partition (/boot/efi)	FAT32; required for UEFI ESP partitions
xfs	Large files, databases	High performance; good for large filesystems
btrfs	Snapshots, subvolumes	Copy-on-write; supports inline RAID
swap	Swap space	Not a filesystem; used for virtual memory
zfs	Enterprise / advanced	Requires ZFS pool via separate zpool config
ntfs	Windows interop	Read/write support via ntfs-3g
fat32	Legacy compatibility	Alias for vfat

5.3 Examples

```
- id: efi_format
  type: format
  volume: efi_part
  fstype: vfat
  label: EFI

- id: root_format
  type: format
  volume: root_part
  fstype: ext4
  label: ubuntu-root
```

```
- id: swap_format
  type: format
  volume: swap_part
  fstype: swap
```

6. type: mount

A mount object attaches a formatted filesystem to a path in the target system's directory tree.

6.1 All Fields

Field	Type	Required	Description
id	string	Yes	Unique identifier
type	string	Yes	Must be "mount"
device	string	Yes	ID of the format object to mount
path	string	Yes	Mount point e.g. /, /boot, /boot/efi, /home
options	string	No	Comma-separated mount options written to /etc/fstab

6.2 Common Mount Options

Option	Effect
defaults	Standard defaults (rw, suid, dev, exec, auto, nouser, async)
noatime	Skip atime updates — improves performance, reduces SSD wear
relatime	Update atime only if mtime is more recent (default on most kernels)
discard	Enable TRIM for SSDs — pass trim commands on file deletion
errors=remount-ro	Remount read-only on ext4 error (recommended for root)
compress=zstd:1	btrfs inline compression with zstd level 1
subvol=@	btrfs subvolume mount
x-systemd.automount	Automount via systemd on first access
nofail	Continue boot even if mount fails (useful for optional data drives)

6.3 Examples

```
- id: efi_mount
```

```
    type: mount
    device: efi_format
    path: /boot/efi

  - id: root_mount
    type: mount
    device: root_format
    path: /
    options: defaults,noatime,errors=remount-ro

  - id: home_mount
    type: mount
    device: home_format
    path: /home
    options: defaults,noatime
```

7. LVM Configuration

Logical Volume Manager (LVM) requires two object types: lvm_volgroup to create the Volume Group, and lvm_partition to create Logical Volumes within it.

7.1 lvm_volgroup Fields

Field	Type	Required	Description
id	string	Yes	Unique identifier
type	string	Yes	Must be "lvm_volgroup"
name	string	Yes	VG name e.g. ubuntu-vg
devices	list	Yes	List of IDs of physical volumes (partitions or RAID arrays)
preserve	boolean	No	true = do not modify existing VG
wipe	string	No	Wipe PV signatures before use

7.2 lvm_partition Fields

Field	Type	Required	Description
id	string	Yes	Unique identifier
type	string	Yes	Must be "lvm_partition"
volgroup	string	Yes	ID of the parent lvm_volgroup
name	string	Yes	LV name e.g. ubuntu-lv

size	string	Yes	Size: 50G, 100%, or -1 for all remaining space
preserve	boolean	No	true = do not modify existing LV
wipe	string	No	Wipe before use

7.3 Complete LVM Example

```
storage:
  config:
  # ── Physical disk ──────────────────────────────────────────
  - id: disk0
    type: disk
    ptable: gpt
    match:
      size: largest
    grub_device: true
    wipe: superblock

  # ── Partitions ─────────────────────────────────────────────
  - id: efi_part
    type: partition
    device: disk0
    size: 512M
    flag: boot

  - id: lvm_pv_part
    type: partition
    device: disk0
    size: -1
    wipe: superblock

  # ── LVM Volume Group ───────────────────────────────────────
  - id: vg0
    type: lvm_volgroup
    name: ubuntu-vg
    devices:
      - lvm_pv_part

  # ── Logical Volumes ────────────────────────────────────────
  - id: lv_root
    type: lvm_partition
    volgroup: vg0
    name: ubuntu-lv
    size: 40G

  - id: lv_swap
    type: lvm_partition
    volgroup: vg0
```

```
    name: swap
    size: 4G

  - id: lv_home
    type: lvm_partition
    volgroup: vg0
    name: home
    size: -1

  # ── Filesystems ──────────────────────────────────────────────
  - id: efi_fmt
    type: format
    volume: efi_part
    fstype: vfat

  - id: root_fmt
    type: format
    volume: lv_root
    fstype: ext4

  - id: swap_fmt
    type: format
    volume: lv_swap
    fstype: swap

  - id: home_fmt
    type: format
    volume: lv_home
    fstype: ext4

  # ── Mounts ───────────────────────────────────────────────────
  - id: efi_mnt
    type: mount
    device: efi_fmt
    path: /boot/efi

  - id: root_mnt
    type: mount
    device: root_fmt
    path: /
    options: defaults,noatime

  - id: home_mnt
    type: mount
    device: home_fmt
    path: /home
    options: defaults,noatime
```

8. type: raid

A raid object creates a Linux MD (Multiple Device) software RAID array from a list of disks or partitions.

8.1 All Fields

Field	Type	Required	Description
id	string	Yes	Unique identifier
type	string	Yes	Must be "raid"
name	string	Yes	MD device name e.g. md0
raidlevel	string	Yes	RAID level: 0, 1, 5, 6, 10
devices	list	Yes	IDs of member disks or partitions
spare_devices	list	No	IDs of hot-spare devices
preserve	boolean	No	true = use existing array
wipe	string	No	Wipe superblock before assembly
ptable	string	No	Add partition table to RAID device (for further partitioning)
metadata	string	No	MD metadata version e.g. 1.2 (default) or 0.90

8.2 RAID Level Summary

Level	Min Disks	Fault Tolerance	Notes
RAID 0	2	None (no redundancy)	Maximum performance; total loss if any disk fails
RAID 1	2	N-1 disks can fail	Mirroring; 50% usable capacity
RAID 5	3	1 disk can fail	Distributed parity; (N-1) usable capacity
RAID 6	4	2 disks can fail	Dual parity; (N-2) usable capacity
RAID 10	4	1 per mirror pair	Stripe of mirrors; 50% usable capacity

8.3 RAID 1 Example (Mirror)

```
storage:
  config:
  - id: disk0
    type: disk
    ptable: gpt
```

```
    match:
      path: /dev/sda
    grub_device: true
    wipe: superblock

  - id: disk1
    type: disk
    ptable: gpt
    match:
      path: /dev/sdb
    grub_device: true
    wipe: superblock

  - id: disk0_efi
    type: partition
    device: disk0
    size: 512M
    flag: boot

  - id: disk1_efi
    type: partition
    device: disk1
    size: 512M
    flag: boot

  - id: disk0_data
    type: partition
    device: disk0
    size: -1
    wipe: superblock

  - id: disk1_data
    type: partition
    device: disk1
    size: -1
    wipe: superblock

  - id: md0
    type: raid
    name: md0
    raidlevel: 1
    devices:
      - disk0_data
      - disk1_data

  - id: root_fmt
    type: format
    volume: md0
    fstype: ext4
```

```
  - id: root_mnt
    type: mount
    device: root_fmt
    path: /
```

9. type: bcache

bcache allows an SSD to act as a cache for a slower HDD, transparent to the filesystem layer.

9.1 Fields

Field	Type	Required	Description
id	string	Yes	Unique identifier
type	string	Yes	Must be "bcache"
backing_device	string	Yes	ID of the slow backing device (HDD partition or disk)
cache_device	string	Yes	ID of the fast cache device (SSD partition or disk)
cache_mode	string	No	writeback (default), writethrough, writearound, none
preserve	boolean	No	Preserve existing bcache

9.2 Example

```
- id: hdd_disk
  type: disk
  match:
    ssd: false

- id: ssd_disk
  type: disk
  match:
    ssd: true

- id: hdd_part
  type: partition
  device: hdd_disk
  size: -1

- id: ssd_cache
  type: partition
  device: ssd_disk
  size: 50G
```

```
- id: cached_device
  type: bcache
  backing_device: hdd_part
  cache_device: ssd_cache
  cache_mode: writeback

- id: data_fmt
  type: format
  volume: cached_device
  fstype: ext4
```

10. ZFS Configuration

ZFS support in autoinstall uses the layout: shorthand or requires the zfs object type available in Ubuntu 20.04+. Full ZFS configuration uses a pool-based approach.

10.1 ZFS via layout Shorthand

```
storage:
  layout:
    name: zfs
  # Optional: specify which disk
  # config: []
```

10.2 ZFS Object Fields

Field	Type	Required	Description
id	string	Yes	Unique identifier
type	string	Yes	Must be "zfs"
pool	string	Yes	ZFS pool name e.g. rpool
vdevs	list	Yes	Virtual device configuration
mountpoint	string	No	Root mount path
pool_properties	mapping	No	ZFS pool properties e.g. ashift: 12
fs_properties	mapping	No	Filesystem properties e.g. compression: lz4

11. Swap Configuration

Swap can be configured as a dedicated partition or as a swap file. The swap: key at the top level of autoinstall configures swap file creation independently of the storage: block.

11.1 Swap Partition (recommended for LVM)

```
- id: swap_part
  type: partition
  device: disk0
  size: 4G

- id: swap_fmt
  type: format
  volume: swap_part
  fstype: swap

- id: swap_mnt
  type: mount
  device: swap_fmt
  path: none
```

11.2 Swap File via swap: Key

```
autoinstall:
  version: 1
  swap:
    size: 0           # 0 = disable swap file (use partition swap instead)
  # OR
  swap:
    size: 4G          # Create a 4 GiB swap file at /swapfile
```

12. Complete Configuration Examples

12.1 Simple UEFI Single Disk

```
autoinstall:
  version: 1
  storage:
    config:
    - id: disk0
      type: disk
      ptable: gpt
      match:
        size: largest
      grub_device: true
      wipe: superblock
```

```
    - id: efi_part
      type: partition
      device: disk0
      size: 512M
      flag: boot

    - id: swap_part
      type: partition
      device: disk0
      size: 4G

    - id: root_part
      type: partition
      device: disk0
      size: -1

    - id: efi_fmt
      type: format
      volume: efi_part
      fstype: vfat
      label: EFI

    - id: swap_fmt
      type: format
      volume: swap_part
      fstype: swap

    - id: root_fmt
      type: format
      volume: root_part
      fstype: ext4
      label: ubuntu-root

    - id: efi_mnt
      type: mount
      device: efi_fmt
      path: /boot/efi

    - id: root_mnt
      type: mount
      device: root_fmt
      path: /
      options: defaults,noatime
```

12.2 RAID 1 with LVM on Top

```
storage:
  config:
```

```
  # Two disks for mirroring
  - id: sda
    type: disk
    ptable: gpt
    match: { path: /dev/sda }
    grub_device: true
    wipe: superblock

  - id: sdb
    type: disk
    ptable: gpt
    match: { path: /dev/sdb }
    grub_device: true
    wipe: superblock

  # EFI on both (UEFI can only use one, but mirrored is safer)
  - { id: sda_efi, type: partition, device: sda, size: 512M, flag: boot }
  - { id: sdb_efi, type: partition, device: sdb, size: 512M, flag: boot }

  # Data partitions to mirror
  - { id: sda_md, type: partition, device: sda, size: -1, wipe: superblock }
  - { id: sdb_md, type: partition, device: sdb, size: -1, wipe: superblock }

  # RAID 1 array
  - id: md0
    type: raid
    name: md0
    raidlevel: 1
    devices: [ sda_md, sdb_md ]

  # LVM on top of RAID
  - id: vg0
    type: lvm_volgroup
    name: vg0
    devices: [ md0 ]

  - { id: lv_root, type: lvm_partition, volgroup: vg0, name: root, size: 40G }
  - { id: lv_swap, type: lvm_partition, volgroup: vg0, name: swap, size: 8G }
  - { id: lv_home, type: lvm_partition, volgroup: vg0, name: home, size: -1 }

  # Filesystems
  - { id: efi_fmt,  type: format, volume: sda_efi, fstype: vfat }
  - { id: root_fmt, type: format, volume: lv_root,  fstype: ext4 }
  - { id: swap_fmt, type: format, volume: lv_swap,  fstype: swap }
  - { id: home_fmt, type: format, volume: lv_home,  fstype: ext4 }

  # Mounts
  - { id: m_efi,  type: mount, device: efi_fmt,  path: /boot/efi }
  - { id: m_root, type: mount, device: root_fmt, path: / }
  - { id: m_home, type: mount, device: home_fmt, path: /home }
```

13. preserve and wipe Options

These two fields control whether existing data is kept or destroyed during installation.

13.1 preserve

When set to true on any storage object, the installer will not modify that object. The disk/partition/filesystem must already exist exactly as specified. Use this for dual-boot setups or when adding Ubuntu alongside existing data.

> **WARNING:** All objects in the dependency chain must also have preserve: true. For example, to preserve a partition, its parent disk must also be preserved.

13.2 wipe Values

Value	Behavior
superblock	Erase filesystem superblock and LVM/RAID signatures (fast)
zero	Write zeros to the entire device (thorough, slow on large disks)
random	Write random data to the entire device (most secure; very slow)
null / omitted	Do not wipe; may fail if existing signatures are detected

14. layout: Shorthand

For simple use cases, the layout: key provides predefined disk configurations. It cannot be combined with config: in the same storage block.

Name	Description	Notes
lvm	EFI + /boot + LVM VG with root LV	Default installer layout; suitable for most servers
direct	EFI + single root partition (no LVM)	Simple; cannot resize volumes later
zfs	EFI + ZFS root pool	Requires zfsutils-linux; experimental in some releases

```
# Simplest possible storage config
```

```yaml
storage:
  layout:
    name: lvm

# Direct layout with explicit disk
storage:
  layout:
    name: direct
  config:
  - id: disk0
    type: disk
    ptable: gpt
    match:
      path: /dev/nvme0n1
    grub_device: true
```

15. Object Ordering Rules

The config: list must be ordered so that dependencies are declared before the objects that reference them. The correct dependency order is:

1. disk objects
2. partition objects (reference disk)
3. raid / bcache objects (reference partitions)
4. lvm_volgroup objects (reference partitions or raid)
5. lvm_partition objects (reference lvm_volgroup)
6. format objects (reference any block device)
7. mount objects (reference format objects)

NOTE: The / (root) mount must appear before any sub-mounts such as /home or /boot, even though they reference different format objects.

16. Common Errors & Troubleshooting

Error / Symptom	Likely Cause & Fix
No bootable device found	grub_device: true missing on disk or EFI partition; check flag: boot on ESP
Installer ignores storage: config	Syntax error in YAML; run yaml-lint or python3 -c 'import yaml; yaml.safe_load(open("autoinstall.yaml"))'
Disk not matched / NotFound error	match: criteria too strict or wrong path; use match: { size: largest } or verify serial

Partition table exists error	wipe: superblock not set on disk, or preserve: false conflict with existing table
LVM VG already exists	Previous install left VG; add wipe: superblock on the PV partition
EFI boot fails after install	ESP not formatted as vfat, flag: boot missing, or grub_device on wrong object
RAID degraded immediately	Disk count mismatch; verify all member IDs resolve to real, distinct devices
size: -1 on multiple partitions	Only one partition per disk can use -1; all others need explicit sizes

TIP: To debug: boot the installer in interactive mode, check /var/log/installer/subiquity-debug* and /var/log/curtin-install.log for detailed error messages.

17. Quick Reference Card

Scenario	Key Objects Required
UEFI single disk, ext4	disk (gpt) → partition (boot) → partition → format (vfat) + format (ext4) → mount (/boot/efi) + mount (/)
BIOS single disk, ext4	disk (gpt) → partition (bios_grub, 1M) → partition → format (ext4) → mount (/)
UEFI + LVM	disk → partition (boot) + partition → lvm_volgroup → lvm_partition(s) → format(s) → mount(s)
RAID 1 mirror	disk × 2 → partition × 2 → raid (level:1) → format → mount
RAID 1 + LVM	disk × 2 → partition × 2 → raid → lvm_volgroup → lvm_partition(s) → format(s) → mount(s)
SSD cache (bcache)	hdd_part + ssd_part → bcache → format → mount
ZFS root (simple)	layout: { name: zfs }
Preserve existing data	Add preserve: true to disk, partition, and format objects; omit wipe